

Video and accelerated filters in OpenGL

Tomáš Milet

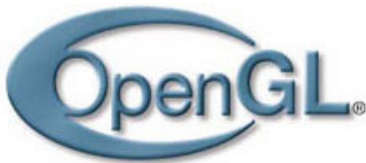
Brno University of Technology, Faculty of Information Technology
Božetěchova 1/2. 602 00 Brno - Královo Pole
login@fit.vutbr.cz



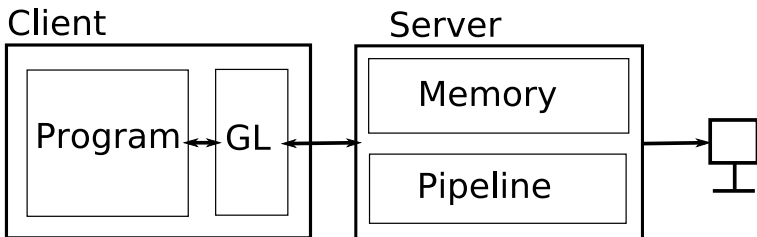
22. dubna 2016

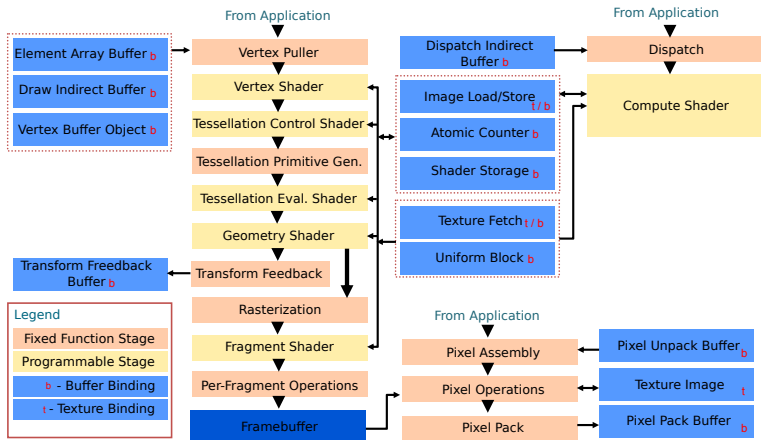
OpenGL

- OpenGL Open Graphics Language (Library)
- OpenGL is API for 3D accelerated graphics
- OpenGL is platform independent
- OpenGL wrapper ported to most languages
- Main goal of OpenGL is to convert scene represented using 3D vectors to 2D rasterized image
- OpenGL can be used for arbitrary computation since OpenGL 4.3
- OpenGL specification contains description of it's GPU language GLSL



- OpenGL can be viewed as client-server architecture
- User application runs on CPU and through OpenGL calls can control GPU





- A scene is, in most cases, rendered into default framebuffer
- Since OpenGL 3.3, it is possible to create custom framebuffers.
- Each framebuffer is composed of a set of color textures and depth and stencil texture
- Framebuffers allow rendering into textures
- Textures do not have to store color but they can store other useful information as well (positions, normals, depths, ...)
- They can be used as intermediate stages in the rendering pipeline

```
//reserve texture id
glGenTextures(1,&tColor);

//bind texture to 2D target
glBindTexture(GL_TEXTURE_2D,tColor);

//set filtering
glTexParameteri(GL_TEXTURE_2D,GL_TEXTURE_MAG_FILTER,GL_NEAREST);
glTexParameteri(GL_TEXTURE_2D,GL_TEXTURE_MIN_FILTER,GL_NEAREST);

//allocate memory on GPU and copy data from CPU
glTexImage2D(GL_TEXTURE_2D,0,GL_RGBA,width,height,0,GL_RGBA,
GL_UNSIGNED_BYTE,data);
```

```
//reserve framebuffer id
glGenFramebuffers(1, &FBO);
//bind framebuffer
glBindFramebuffer(GL_FRAMEBUFFER, FBO);

//set framebuffer attachments
glFramebufferTexture2D(GL_FRAMEBUFFER, GL_COLOR_ATTACHMENT0,
    GL_TEXTURE_2D, tColor, 0); //attach color texture
glFramebufferTexture2D(GL_FRAMEBUFFER, GL_COLOR_ATTACHMENT1,
    GL_TEXTURE_2D, tNormal, 0); //attach normal texture

GLenum H[]={
    GL_COLOR_ATTACHMENT0, //layout(location=0) out vec4 fragColor;
    GL_COLOR_ATTACHMENT1 //layout(location=1) out vec3 fragNormal;
};
glDrawBuffers(2, H); //set set of targets
glBindFramebuffer(GL_FRAMEBUFFER, 0); //unbind framebuffer
```

```
//activate framebuffer
glBindFramebuffer(GL_FRAMEBUFFER, FBO);

//...

//draw some geometry
glDrawArrays(GL_TRIANGLE_STRIP, 0, 4);

//...

//deactivate framebuffer
glBindFramebuffer(GL_FRAMEBUFFER, 0);
```

Data from one framebuffer can be transferred into another framebuffer using blit operation.

```
//activate source framebuffer
glBindFramebuffer(GL_READ_FRAMEBUFFER, FBO0);

//active destination framebuffer
glBindFramebuffer(GL_DRAW_FRAMEBUFFER, FBO1);

//copy color buffers
glBlitFramebuffer(
    srcX0,srcY0,
    srcX2,srcY1,
    dstX0,dstY0,
    dstX1,dstY1,
    GL_COLOR_BUFFER_BIT,
    GL_NEAREST);

//deactivate framebuffers
glBindFramebuffer(GL_READ_FRAMEBUFFER, 0);
glBindFramebuffer(GL_DRAW_FRAMEBUFFER, 0);
```

Video player in OpenGL

- A window has to be created using 3th party libraries (SDL2, GLUT, Qt).
- A video has to be decoded outside of OpenGL using other libraries (OpenCV, ffmpeg, ...).
- Current frame has to be stored in OpenGL texture or buffer in GPU memory.
- A precise time has to be used for correct synchronisation of video frames and rendering frames (std::chrono).
- A shader program has to contain some kind of fullscreen quad program.

OpenGL requires a rendering context. There are 3 main context parameters (version, profile and flags) that can be set using SDL2.

```
SDL_Init(SDL_INIT_VIDEO); //init. video
this->_window = SDL_CreateWindow("sdl2", 0, 0, width, height,
    SDL_WINDOW_OPENGL | SDL_WINDOW_SHOWN);

unsigned profile = SDL_GL_CONTEXT_PROFILE_CORE; //context profile
unsigned flags = SDL_GL_CONTEXT_DEBUG_FLAG; //context flags
SDL_GL_SetAttribute(SDL_GL_CONTEXT_PROFILE_MASK, profile);
SDL_GL_SetAttribute(SDL_GL_CONTEXT_FLAGS, flags);

std::vector<uint32_t> versions = {450, 440, 430, 420, 410, 400, 330};
for (auto x : versions) {
    SDL_GL_SetAttribute(SDL_GL_CONTEXT_MAJOR_VERSION, x/100);
    SDL_GL_SetAttribute(SDL_GL_CONTEXT_MINOR_VERSION, (x%100)/10);
    this->_context = SDL_GL_CreateContext(this->_window); //create context
    if (this->_context != nullptr) break;
}

glewInit(); //initialisation of gl* functions
```

OpenCV library has a lot of functions that deals with images and videos. `cv::VideoCapture` object is able to open video and load frames.

```
#include<opencv2/core/core.hpp>
#include<opencv2/highgui/highgui.hpp>

class Video{
protected:
    cv::Mat bgr_frame;
    cv::VideoCapture _video;
    ...
};

Video::Video(std::string fileName){
    this->_opened=this->_video.open(fileName);
    if(!this->_opened){...}
    this->_frameCount = this->_video.get(CV_CAP_PROP_FRAME_COUNT);
    this->_fps         = this->_video.get(CV_CAP_PROP_FPS);
    this->_width       = this->_video.get(CV_CAP_PROP_FRAME_WIDTH);
    this->_height      = this->_video.get(CV_CAP_PROP_FRAME_HEIGHT);
}
//read current frame and move to next
void* Video::getData(){
    ...
    if(this->_video.read(bgr_frame)) return nullptr;
    return this->bgr_frame.data;
    ...
}
```

std::chrono - C++ std library that provides high precise timers.

```
#include<chrono>
#include<ctime>

template<typename TYPE>
class Timer{
protected:
    std::chrono::time_point<std::chrono::high_resolution_clock>_start;
    std::chrono::time_point<std::chrono::high_resolution_clock>_last ;
public:
    Timer() {
        this->reset();
    }
    void reset() {
        this->_start = std::chrono::high_resolution_clock::now();
        this->_last = this->_start;
    }
    TYPE elapsedFromStart() {
        auto newTime = std::chrono::high_resolution_clock::now();
        std::chrono::duration<TYPE> elapsed = newTime - this->_start;
        this->_last = newTime;
        return elapsed.count();
    }
    ...
};
```

Video frames and window frames has to be correctly synchronized.

```
void draw(Data*data) {
    glClear(GL_COLOR_BUFFER_BIT|GL_DEPTH_BUFFER_BIT);

    ...
    float newTime = data->timer.elapsedFromStart();
    if( (newTime-data->lastFrameTime)>1./data->video->getFps()) {
        if(data->running) {
            glBindTexture(GL_TEXTURE_2D,data->frame->getId());
            glTexImage2D(GL_TEXTURE_2D,0,GL_RGB,
                data->video->getWidth(),
                data->video->getHeight(),0,
                GL_BGR,GL_UNSIGNED_BYTE,data->video->getData());
            glTexParameterf(GL_TEXTURE_2D,GL_TEXTURE_MIN_FILTER,GL_LINEAR);
            glTexParameterf(GL_TEXTURE_2D,GL_TEXTURE_MAG_FILTER,GL_LINEAR);
            data->lastFrameTime = newTime;
        }
    }

    ...
}
```

In order to render a frame, there has to be textured rectangle. There are many ways of creating of rectangle (buffers, geometry shaders, vertex shaders,...). The simplest way employ vertex shaders.

```
#version 450

//texture coordinates
out vec2 vCoord;

void main() {
    //compute texture coordinates using gl_VertexID
    vec2 coord = vec2(gl_VertexID%2,gl_VertexID/2);

    //compute clips space vertex positions
    gl_Position = vec4(-1+2*coord,1,1.);

    //send texture coordinates to fragment shader
    vCoord = vec2(coord.x,1-coord.y);
}

//invoke vertex shader 4 times and construct triangle strip
glDrawArrays(GL_TRIANGLE_STRIP,0,4);
```

```
#version 450

//output for color
layout(location=0)out vec4 fColor;

//texture
layout(binding=0)uniform sampler2D frame;

//texture coordinates
in vec2 vCoord;

void main(){
    //address texture and return color
    fColor = texture(frame,vCoord);
}

//active texture unit 0
glActiveTexture(GL_TEXTURE0);

//bind texture frame to texture unit 0
glBindTexture(GL_TEXTURE_2D, frame);
```

Simple video filters in OpenGL

```
#version 450

layout(location=0) out vec4 fColor;

layout(binding=0) uniform sampler2D frame;

//texture coordinates
in vec2 vCoord;

uniform uvec2 windowSize = uvec2(1,1);

void main() {
    //read from texture
    vec3 color = texture(frame,vCoord).xyz;

    //convertex to intensity
    float intensity = dot(vec3(.2126, .7152, .0722),color);

    //write intensity to framebuffer
    fColor = vec4(intensity);
}
```



<https://www.youtube.com/watch?v=36lSzUMBjnc>

```
#version 450

layout(location=0) out vec4 fColor;

layout(binding=0) uniform sampler2D frame;

//texture coordinates
in vec2 vCoord;

uniform uvec2 windowSize = uvec2(1,1);

void main() {
    //read from texture
    vec3 color = texture(frame,vCoord).xyz;

    //convertex to intensity
    float intensity = dot(vec3(.2126, .7152, .0722),color);

    //write intensity to framebuffer
    fColor = vec4(intensity>.4);
}
```



<https://www.youtube.com/watch?v=36lSzUMBjnc>

```
#version 450

layout(location=0) out vec4 fColor;

layout(binding=0) uniform sampler2D frame;

//texture coordinates
in vec2 vCoord;

uniform uvec2 windowSize = uvec2(1,1);

void main() {
    //read from texture
    vec3 color = texture(frame,vCoord).xyz;

    float redness = 1-distance(color.rgb,vec3(1,0,0))/sqrt(3);

    //convertex to intensity
    float intensity = dot(vec3(.2126,.7152,.0722),color);

    //write color to framebuffer
    fColor = vec4(mix(vec3(intensity>0.4),redGradient(intensity).xyz,
        smoothstep(0.6,.8,redness,1)),1);
}
```



<https://www.youtube.com/watch?v=36lSzUMBjnc>

```
#version 450
...
void main() {
    //read from texture
    float intensities[9];

    for(int y=-1;y<=1;++y)
        for(int x=-1;x<=1;++x)
            intensities[(1+y)*3+(1+x)] =
                dot(vec3(.2126,.7152,.0722),texelFetch(frame,texelCoord+ivec2(x,y),0).xyz);

    vec3 color = texelFetch(frame,texelCoord,0).xyz;

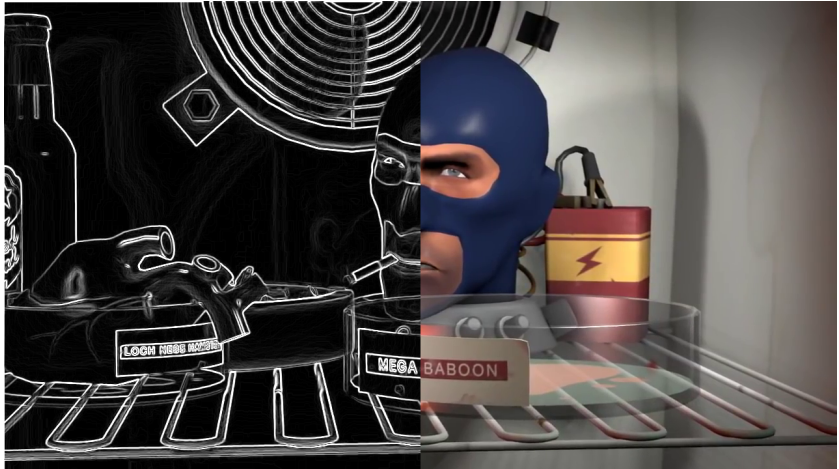
    float xSobelMask[9] = {-1,0,1,-2,0,2,-1,0,1};
    float ySobelMask[9] = {-1,-2,-1,0,0,0,1,2,1};

    float xSobel = 0;
    float ySobel = 0;

    for(int i=0;i<9;++i){
        xSobel += intensities[i]*xSobelMask[i];
        ySobel += intensities[i]*ySobelMask[i];
    }

    vec3 sobel = vec3(length(vec2(xSobel,ySobel)));

    fColor = vec4(sobel,1);
}
```



<https://www.youtube.com/watch?v=36lSzUMBjnc>

#version 450

```
float gaussFunction(float x, float sigma){
    return inversesqrt(2*radians(180)*sigma*sigma)*exp(-x*x/(2*sigma*sigma));
}

void main(){

    const int gaussSize = 10;
    vec3 gauss = vec3(0);
    float accumulatedWeights = 0;
    for(int x=-gaussSize;x<=gaussSize;++x)
        for(int y=-gaussSize;y<=gaussSize;++y){
            float distanceFromCenter = length(vec2(x,y));
            float g = gaussFunction(distanceFromCenter,10);
            accumulatedWeights += g;
            gauss += texelFetch(frame, texelCoord+ivec2(x,y), 0).xyz*g;
        }
    gauss /= accumulatedWeights;

    fColor = vec4(gauss,1);
}
```



<https://www.youtube.com/watch?v=36lSzUMBjnc>



<https://www.youtube.com/watch?v=36lSzUMBjnc>

- <http://www.opengl.org/sdk/docs/>
- <http://www.opengl.org/documentation/glsl/>
- <https://wiki.libsdl.org/FrontPage>
- http://docs.opencv.org/3.1.0/d8/dfe/classcv_1_1VideoCapture.html
- <http://en.cppreference.com/w/cpp/chrono>

Thank you for your attention!